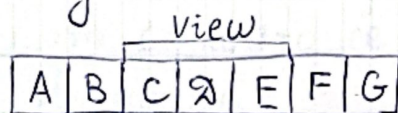


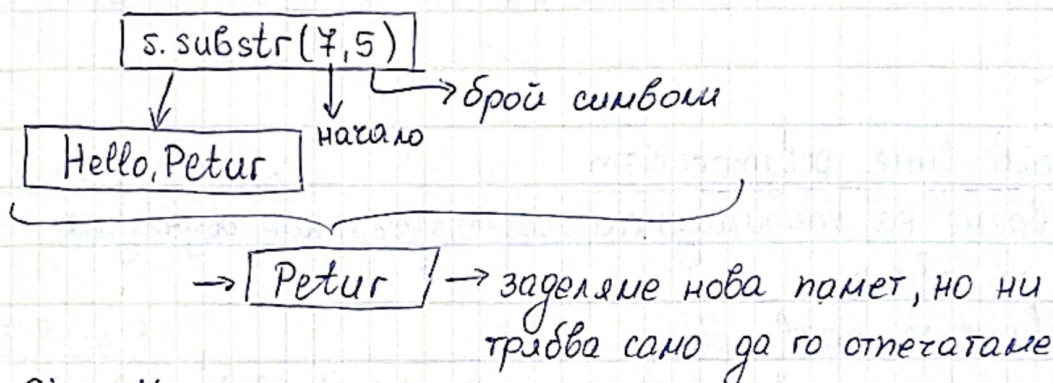
Изглед (View)

def. Клас, който се използва за преглед на интервал от колекция.



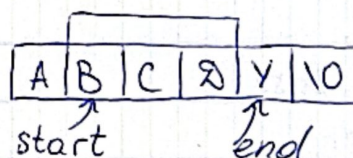
↳ искаме да имаме константен достъп

Пример: `String s = "Hello, Petar"`



StringView - изглед на част от стринг

1 начин: Указател към началото
Указател към края



2 начин: Указател към началото
Дължина

Полиморфизъм

def. Едно име на функция, много различни имплементации

Пример:

Function overloading
f() {...}
f(int) {...}
f(int, int) {...}

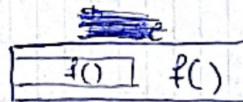
Видове

1. Compile-time polymorphism

- по време на компиляцията се определя коя функция да се извика
- function overloading
- operator overloading

При ООП:

```
class Base {  
public:  
    f() { cout << "Base::f()"; }  
};  
class Der: public Base {  
public:  
    f() { cout << "Der::f()"; }  
};
```



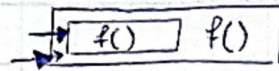
```
Der d;  
d.f();      Der::f()  
K(d);      Base::f()  
s(d);      Der::f()
```

```
K(Base* ptr);  
ptr -> f();  
S(Der* ptr);  
ptr -> f();
```

Статично свързване

→ изборът на функцията става по време на компилация

→ определя се от: типа на указателя / референцията, от който се извиква функцията



```
Der d;  
Der* ptr=d;  
ptr->f() // Der::f()
```

```
Base* ptr2=d;  
ptr2->f(); // Base::f()
```

Динамично свързване

→ изборът коя функция да се извика става по време на изпълнение

2. Run-time polymorphism

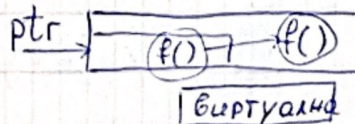
→ чрез виртуални функции

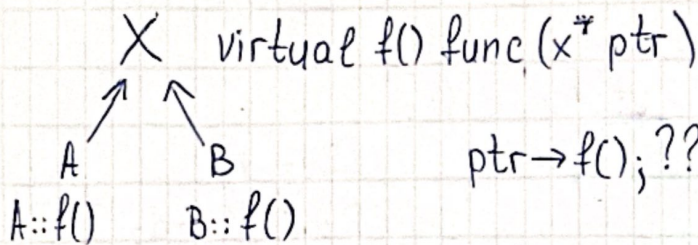
```
class Base {  
    virtual void f() { cout << "Base:: f()"; }  
}
```

```
class Der : Base {  
    void f() { cout << "Der:: f()"; }  
}
```

```
Der d;  
d.f(); // Der::f()
```

```
Base* ptr = &d;  
ptr->f(); // Der::f()
```





ptr → f(); ??? Не знаем какво ще се извика
 A::f(); B::f(); (x::f())
 x* temp = (rand() % 2 == 0) ? new A();
 new B();
 func(temp);

```

Base {
  virtual void f() { cout << "Base::f()"; }
  ↑

```

```

Der
void f() override

```

винаги го пишем, защото е по-четливо
и прави проверка на синтаксиса

презаписване
виртуална функция

при грешка в синтаксиса
→ компилационна грешка

Пример:

```

Base
virtual f() { Base::f(); }
Base() { f(); } // Base::f()
Der още не е създадена
~Base() { f(); } // Base::f()
g() { f(); }
Der вече е изтрил

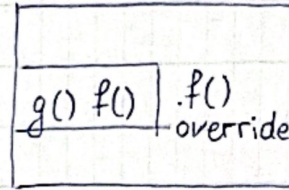
Der: Base {
  f() override { Der::f(); }
};

```

```

{ Der d;    // Base::f()
  d.f();    // Der::f()
  Base* ptr = &obj;
  ptr->g(); // Der::f()
} Base::f()

```



def. чисто виртуална функция (pure virtual) -
 виртуална функция, която няма имплементация
 ⇒ предназначена да бъде презаписана от
 наследниците

```

class Base {
  virtual void f() = 0;
}

```

A
 f() override { ... }

B
 f() override { ... }

в случая не можем да
 правим обекти от
 тип Base

def. Абстрактен клас -
 клас, който има поне една чисто
 виртуална функция
 - предназначен за наследяване

С не override-ва ⇒ С също е абстрактен

Пример: Shapes

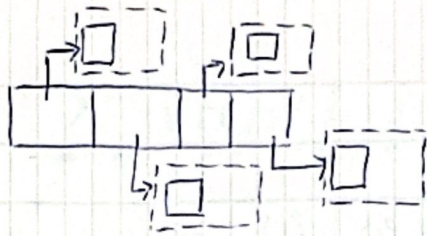
```

Shape / Polygon
virtual getArea() = 0;

```

getArea() override

-масив от shape; трябва
 да е колекция от
 указатели, защото Δ, O, □
 са различни обекти
 и имат различна
 големина



- виртуален деструктор



При полиморфна йерархия винаги деструктора, на базовия клас трябва да е виртуален

```
virtual ~Shape => ~rect() {
    } ~shape {}
```

class Base - когато няма да разписваме деструктора, но
искаме да е virtual
int x;
virtual ~Base = default;

Ключова дума: final — final за виртуална
функции

void f() override (final)

↳ не може да се презаписва
надолу

- final за класове - не може да се
наследява класа